📊 SQLAlchemy 😸

1. SQLAlchemy 소개

SQLAlchemy는 Python에서 데이터베이스를 보다 편리하고 객체지향적으로 다룰 수 있도록 도와주는 ORM 라이브러리이다. ORM(Object Relational Mapping)은 테이블을 클래스, 레코드를 객체처럼 다룰 수 있게 해주어 SQL 문을 직접 작성하지 않고도 CRUD 작업이 가능하도록 한다. SQLAlchemy는 두 가지 스타일을 지원하는데, Core는 SQL에 가까운 저수준 방식이고, ORM은 Python 클래스를 기반으로 모델을 정의해 고수준으로 데이터를 다룬다.

- Python의 대표 ORM(Object Relational Mapping)
- DB 테이블을 파이썬 클래스처럼 다룰 수 있음
- Core(저수준 SQL 작성) + ORM(모델 기반)

---

2. SQLAlchemy 설치

```
pip install sqlalchemy
pip install pymysql # MySQL 사용 시
```

---

3. 기본 구조

SQLAlchemy의 ORM 구조는 크게 Engine, Session, Declarative Base로 구성된다.

- **Engine**: DB와의 연결을 관리하며 SQL 실행의 출발점이다.
- **Session**: ORM을 이용한 DB 작업의 트랜잭션 단위를 담당한다. commit/rollback이 필요한 이유도 Session 때문이다.
- **Declarative Base**: ORM 모델을 정의하기 위한 기반 클래스로, 모든 모델이 이 Base를 상속한다.

이 세 가지 구조를 이해하면 ORM 사용 흐름이 명확해진다.

SQLAlchemy는 여러 컴포넌트가 함께 동작하는 구조로 이루어져 있다:

- **Engine**: 데이터베이스와의 실제 연결을 담당한다. 커넥션 풀링, DB URL 설정 등을 담당한다.
- **Session**: ORM을 사용하여 DB와 상호작용할 때 필요한 트랜잭션 단위 작업을 관리한다. Session은 하나의 작업 단위를 나타내며, commit 또는 rollback을 명확히 해줘야 한다.
- **Declarative Base(Model)**: ORM에서 사용할 모델(테이블 구조)을 정의하는 데 사용된다. Base 클래스를 상속한 Python 클래스는 실제 DB 테이블과 매핑된다.

이 세 요소는 SQLAlchemy ORM을 사용할 때 반드시 이해해야 하는 핵심 구성 요소이다.

SQLAlchemy는 보통 다음 요소로 구성됨:

- **Engine**: DB 연결 담당
- **Session**: 트랜잭션 단위 작업 담당
- **Declarative Base(Model)**: ORM 모델 정의

4. Engine 생성

Engine은 SQLAlchemy에서 **DB와의 실제 연결을 관리하는 핵심 객체**이다. ORM이든 Core든 모든 DB 작업은 Engine을 기반으로 수행된다.

- DB URL을 분석해서 해당 DB 드라이버를 통해 연결을 설정한다.
- 내부적으로 커넥션 풀(Connection Pool)을 관리한다.
- SQL 실행은 Engine 또는 Engine에서 얻은 Connection을 통해 이루어진다.

Engine은 실제 SQL 실행의 시작점이라고 보면 된다.

```
from sqlalchemy import create_engine

engine = create_engine(
    "mysql+pymysql://user:password@localhost:3306/mydb",
    echo=True   # SQL 로그 출력
)
```

---

5. Declarative Base 선언

Declarative Base는 SQLAlchemy ORM에서 **모델 클래스를 정의하기 위한 기반 클래스**이다. 이 Base를 상속한 클래스는 자동으로 데이터베이스 테이블과 매핑되며, SQLAlchemy는 이를 기반으로 테이블 생성, 매핑, 쿼리 등을 수행할 수 있게 된다.

- Base는 ORM의 시작점이며 모든 모델의 부모 클래스 역할을 한다.
- `Base.metadata`를 통해 모든 테이블 정보가 수집되고 `create_all()` 등을 통해 실제 테이블이 생성된다.
- Base를 사용하면 클래스를 테이블처럼 자연스럽게 쓸 수 있다.

```
from sqlalchemy.orm import declarative_base

Base = declarative_base()
```

---

6. ORM 모델 정의

ORM 모델은 Python 클래스를 통해 DB 테이블을 직접 정의하는 방식이다. 이 클래스를 기반으로 SQLAlchemy는 컬럼, 제약조건 등을 분석하고 테이블 구조를 구성한다.

모델 정의 시 중요한 개념:

- `Column()`: 테이블의 실제 컬럼을 표현
- `nullable`, `unique` 등 컬럼 제약조건을 코드에서 표현 가능
- `func.now()`처럼 DB의 내장 함수를 활용해 기본값 설정 가능

- ORM 모델은 객체 조작만으로도 INSERT, UPDATE, DELETE가 가능해진다.

```
from sqlalchemy import Column, Integer, String, DateTime, func

class User(Base):
    __tablename__ = "users"

    id = Column(Integer, primary_key=True)
    name = Column(String(100), nullable=False)
    email = Column(String(200), unique=True, nullable=False)
    created_at = Column(DateTime, default=func.now())
```

ORM 모델은 DB 테이블 구조를 Python 클래스 형태로 정의하는 것이다. 각 클래스는 테이블, 각 속성은 컬럼을 나타낸다. SQLAlchemy는 이 모델을 기준으로 자동으로 SQL을 생성하고 실행한다.

- `__tablename__`: 해당 모델이 매핑될 테이블 이름
- `Column()`: 컬럼 정의 (타입, 길이, 제약조건 등 설정 가능)
- `primary_key=True`: 기본키 설정
- `default=func.now()`: SQLAlchemy의 func를 사용하면 DB 함수(NOW(), CURRENT_TIMESTAMP 등)를 활용할 수 있음

이 방식 덕분에 SQL 없이도 객체를 조작하는 것만으로 DB를 제어할 수 있다.

```
from sqlalchemy import Column, Integer, String, DateTime, func

class User(Base):
    __tablename__ = "users"

    id = Column(Integer, primary_key=True)
    name = Column(String(100), nullable=False)
    email = Column(String(200), unique=True, nullable=False)
    created_at = Column(DateTime, default=func.now())
```

---

7. 테이블 생성

```
Base.metadata.create_all(engine)
```

---

8. Session 생성

Session은 SQLAlchemy ORM에서 **트랜잭션 단위로 DB와 상호작용하는 핵심 객체**이다. 모든 CRUD 작업은 반드시 Session을 통해 수행된다.

- Session은 하나의 작업 단위를 의미하며, commit 또는 rollback으로 종료된다.
- DB 연결은 Engine이 담당하고, Session은 그 위에서 ORM을 이용해 데이터를 읽고 쓰는 역할을 한다.

- Session을 직접 생성하는 대신 `sessionmaker()` 팩토리를 사용해 Session 클래스를 생성하고, 이를 통해 session 인스턴스를 만든다.

`sessionmaker()`는 다음 역할을 한다:

- Session 클래스를 생성하는 팩토리
- Engine과 연결된 Session 객체를 자동으로 생성해준다
- 필요할 때마다 독립적인 Session 인스턴스를 만들 수 있게 한다

```
from sqlalchemy.orm import sessionmaker

SessionLocal = sessionmaker(bind=engine)
session = SessionLocal()
```

---

9. CRUD 기본

SQLAlchemy ORM에서는 객체 기반으로 CRUD를 수행한다. Session을 통해 DB에 접근하며 commit을 통해 변경사항을 DB에 반영한다.

- **CREATE**: 객체를 생성하고 `session.add()` 후 `commit()`
- **READ**: `session.query()`를 사용해 객체 조회
- **UPDATE**: 조회한 객체의 값을 변경 후 commit
- **DELETE**: `session.delete(object)` 후 commit

ORM은 SQL 문을 직접 작성하지 않아도 객체를 조작하는 것만으로 DB 작업을 수행할 수 있다는 장점이 있다.

SQLAlchemy ORM은 객체를 조작하는 방식으로 CRUD(Create, Read, Update, Delete)를 처리한다. Session을 통해 객체를 저장(add), 조회(query), 수정, 삭제(delete)할 수 있다. SQL을 직접 작성하는 방식과 달리 Python 코드로 수행할 수 있어 가독성이 높고 유지 관리가 쉬워진다.

CREATE

```
new_user = User(name="swan", email="swan@example.com")
session.add(new_user)
session.commit()
```

READ

```
users = session.query(User).all()
user = session.query(User).filter(User.email == "swan@example.com").first()
```

UPDATE

```
user = session.query(User).filter(User.id == 1).first()
user.name = "new name"
session.commit()
```

DELETE

```
session.delete(user)
session.commit()
```

---

10. 필터링 & 조건 검색

```
session.query(User).filter(User.name.like("%a%"))
session.query(User).filter(User.id.in_([1, 2, 3]))
session.query(User).filter(User.created_at >= "2024-01-01")
```

---

11. 정렬 & 페이징

```
session.query(User).order_by(User.created_at.desc()).limit(10).offset(20)
```

---

12. 관계 설정 (1\:N)

관계 설정은 ORM에서 테이블 간 연관 관계를 표현하는 기능이다. SQLAlchemy는 ForeignKey와 relationship을 이용해 관계를 구성한다.

핵심 개념:

- **ForeignKey**: DB 레벨에서 실제 관계를 연결
- **relationship()**: Python 객체 간의 참조 관계를 구성
- **back_populates**: 양방향 관계 설정 시 서로 연결되는 이름 지정

이 설정을 통해 ORM은 조인 없이도 객체 탐색으로 관련 데이터를 조회할 수 있다.

```
from sqlalchemy.orm import relationship
from sqlalchemy import ForeignKey

class Post(Base):
    __tablename__ = "posts"
```

```python
    id = Column(Integer, primary_key=True)
    title = Column(String(200))
    user_id = Column(Integer, ForeignKey("users.id"))

    user = relationship("User", back_populates="posts")

class User(Base):
    __tablename__ = "users"

    id = Column(Integer, primary_key=True)
    name = Column(String(100))

    posts = relationship("Post", back_populates="user")
```
(1:N)

관계 설정은 테이블 간 연관관계를 ORM 모델로 표현하는 과정이다. SQLAlchemy는 ForeignKey와 relationship을 조합하여 관계를 정의한다.

- **ForeignKey**: 관계를 연결하는 실제 컬럼
- **relationship()**: Python 객체 간 연관 관계 설정
- **back_populates**: 양방향 관계 설정 시 서로 연결되는 속성 이름

이 설정을 통해 `user.posts` 또는 `post.user`처럼 객체 관계 탐색이 가능해진다. 이는 ORM의 큰 장점 중 하나이다. (1\:N)

```python
from sqlalchemy.orm import relationship
from sqlalchemy import ForeignKey

class Post(Base):
    __tablename__ = "posts"

    id = Column(Integer, primary_key=True)
    title = Column(String(200))
    user_id = Column(Integer, ForeignKey("users.id"))

    user = relationship("User", back_populates="posts")

class User(Base):
    __tablename__ = "users"

    id = Column(Integer, primary_key=True)
    name = Column(String(100))

    posts = relationship("Post", back_populates="user")
```

---

13. 트랜잭션 관리

```
try:
    session.add(obj)
    session.commit()
except:
    session.rollback()
    raise
finally:
    session.close()
```

---

14. FastAPI 연동 기본

```
from fastapi import Depends

def get_db():
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()
```

---

15. 고급 기능

- One-to-One / Many-to-Many
- Async SQLAlchemy
- Alembic 마이그레이션
- Eager / Lazy Loading 동작 방식
- Bulk Insert

---

16. 필수 기억 정리

- Session은 반드시 닫는다
- commit, rollback 관리 중요
- ORM 관계 설정 시 back_populates 또는 backref 필수
- 모델 변경 시에는 Alembic 권장